From Semantics to Decisions: A Federated Vision-Language Decision Model for UAV Power-Line Inspection

Mahdi Khashan

[a][Affiliation to be added],

Abstract

A recent agenda for edge-native semantic UAV inspection shows that a lightweight vision-language model (VLM) can run on a Jetson-class drone computer, but needs 2.8 s per image; it leaves open how to make onboard semantics fast, quantitatively reliable, trainable without synthetic captions, and collaborative across a fleet. We address these open problems with a vision-language decision model, laya-vision (201M parameters), which answers typed multiple-choice and yes/no inspection questions in one forward pass and returns calibrated probabilities instead of generated text. On 35,415 power-line component images (ElecVQA-Bench, InsPLAD), the published checkpoint fails zero-shot (condition macro F1 0.180, against 0.158 for always answering "good"). Fine-tuning on 4,807 images, with labels from the dataset folders, raises the macro F1 to 0.703, component accuracy to 0.924 and defect AUROC to 0.965, with an expected calibration error of 0.058. Four questions take 74 ms per image on an RTX 2080 Ti; our estimate for a Jetson Orin NX is about 0.67 s. Used as a semantic trigger, the calibrated defect probability passes 15.3% of the images to a slower describing model and finds 88.5% of the defects. We then train it with FedAvg across simulated tower-wise drones. Accuracy falls as the same data is split over more drones (macro F1 0.624, 0.541 and 0.474 for 3, 6 and 18 drones), but stays above drones that train alone (0.371). A start fine-tuned on separate server towers lifts fleet FedAvg from 0.513 to 0.636 macro F1, close to central training, while the fleet images stay on the drones.

Keywords: semantic edge intelligence, vision-language model, decision model, federated learning, UAV inspection, power lines

1. Introduction

Unmanned aerial vehicles (UAVs) inspect bridges, tunnels and power lines faster and more safely than crews on foot. Farahani et al. [1] describe how inspection pipelines move from cloud-centric image analysis, through onboard detectors such as YOLO, to edge-native semantic inspection, in which a lightweight vision-language model (VLM) on the drone interprets what the detector found. Their feasibility study on an NVIDIA Jetson Orin shows that this is possible but slow: a fine-tuned SmolVLM-256M [4] needs about 2.8 s per image, against 0.14 s for YOLO26M, and continuous captioning of a 10 s, 30 fps clip takes about 14 min [1]. Energy is modest relative to the battery; latency is the bottleneck. The same group's EVLM system [2] reports 1.4–6.4 s per 4 s video segment for a LoRA-tuned SmolVLM on power-line inspection, and a companion study profiles cloud, onboard and split deployment of such models [3].

The agenda in [1] lists the system challenges that remain (its Table 1) and the directions that follow. Five of them are the starting point of this paper:

1. Low-latency multimodal reasoning: semantic inference must fit the flight.
2. Hierarchical semantic processing: an inexpensive stage should decide when the expensive VLM runs.
3. Multimodal training data: image–text inspection data is scarce, and the study relies on captions generated by GPT-4o.
4. Trustworthy autonomy: outputs need confidence estimates that a mission planner can act on.
5. Collaborative edge intelligence: many UAVs should improve one model while models run locally on each drone.

The feasibility study also leaves a methodological gap: semantic quality is shown with three qualitative examples, so the gain from fine-tuning is not measured.

Many inspection questions do not need free text. "Does this clamp have a defect?" and "Which condition does this insulator have?" are choices among known options. Decision models answer such typed questions in one forward pass and return a probability for each option. Laya [5] introduced the design for text, and laya-vision [6] replaces its text encoder with a truncated SmolVLM backbone, so that the input can be an image. A decision model is therefore a natural middle stage between a detector and a describing VLM: it keeps the vision-language backbone, but gives labels and probabilities at a cost close to one image encoding.

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

The images that would improve such a model are spread across the fleet that collected them. Different operators may own the drones, and imagery of critical infrastructure is sensitive. Federated learning [9] trains one model across clients without moving their data, but inspection data differs strongly between drones, because each drone flies its own towers.

This paper asks four questions, each tied to an open problem of [1]:

RQ1 How well does a vision-language decision model classify power-line components and their condition, zero-shot and after fine-tuning on labels that need no caption generation? (Challenges 3 and the evaluation gap.)

RQ2 What is its latency, and can its calibrated output act as the trigger of a hierarchical pipeline? (Challenges 1, 2 and 4.)

RQ3 How close does federated fine-tuning across tower-wise drones come to central fine-tuning? (Challenge 5.)

RQ4 Does a model fine-tuned centrally on separate data help the federated fleet? (Challenge 5.)

Our contributions are:

- A gap map from the challenges of [1] to measured evidence (Table 1), including which challenges this work does not address.
- A quantitative evaluation of a vision-language decision model on 35,415 InsPLAD component images, from zero-shot to two rounds of fine-tuning, with ground truth from the dataset folders, bootstrap intervals, calibration error, a class-share correction, and item-level comparison with a fine-tuned ResNet-50 and a LoRA-tuned SmolVLM.
- A latency measurement per image and per segment, and an analysis of the decision model as the semantic trigger of a hierarchical pipeline.
- A federated study with towers as drones: a one-drone control, FedAvg with 3, 6 and 18 drones, local-only training, a base start against a fine-tuned start on the same fleet, and the update size of each trainable part of the model.

2. Background and gap analysis

2.1. The edge-native semantic inspection agenda

Farahani et al. [1] classify UAV inspection into three paradigms by where inference runs: cloud-centric analysis, onboard edge perception with detectors, and edge-native semantic inspection with VLMs. For the third, they list seven requirements: edge sensing and hardware, low-latency multimodal reasoning, hierarchical semantic processing, mission-aware operation, multimodal training data, cross-structure generalization, and trustworthy autonomy. Their outlook adds five directions: agentic inspection, semantic digital twins, collaborative edge intelligence, multimodal sensing, and trustworthy, adaptive reasoning with continual learning. Their experiment uses the COCO-Bridge dataset with GPT-4o captions, YOLO26M for localization and SmolVLM-256M with LoRA for description.

Table 1 maps each open problem to the evidence in this paper. We mark a problem as addressed only when we have a measurement, as partly addressed when the evidence is an estimate or a simulation, and as open otherwise.

2.2. Related work

Edge VLMs for UAV inspection. EVLM [2] filters key frames and runs a LoRA-tuned SmolVLM-256M on a Jetson for power-line inspection. It reduces the uplink of a 4 s segment from 485 kB to 25 kB at a cost of 1.4–6.4 s of generation per segment. The cloud, edge or split study [3] profiles where such a model should run. INAR-VL [7] routes vision-language requests between edge and cloud with accuracy-aware policies. Other semantic inspection systems [1] pair VLMs with large language models or segmentation models, mostly in the cloud. All of these generate text. We study a non-generative model whose outputs are labels and probabilities.

Decision models. Laya [5] scores typed questions (choice, yes/no, score) in one pass with a reinforcement-style objective and a temperature for each question type [8]. Laya-vision [6] adds image input. We use its published checkpoint and its fine-tuning recipe.

Federated learning. FedAvg [9] averages client models weighted by data size. Non-IID client data degrades FedAvg [10], and FedProx [11] adds a proximal term for heterogeneous clients. Flower [12] is a common framework for such simulations. Federated learning has been applied to the routing of language models [13] and to the uncertainty thresholds that decide when an edge model escalates to the cloud [14]. A recent study combines escalation of uncertain detections with federated adapter updates for guardrail inspection [15]. To our knowledge, federated fine-tuning of a vision-language decision model on drone inspection data has not been reported.

3. Method

3.1. Model and questions

Laya-vision [6] (checkpoint thaitea/laya-vision, revision f2fe3c1, 201M parameters) uses SmolVLM-256M cut to 20 of its 30 language layers, followed by a decision head. For an input and a typed question, the head returns one logit per option. A temperature per question type converts the logits to probabilities. The input in all our experiments

Table 1: Open problems of the edge-native semantic inspection agenda [1] and the evidence in this paper. "Est." marks a number derived from a hardware factor, not measured on a Jetson.

Open problem in [1]	What [1] reports	What this paper adds	Status
Low-latency reasoning	SmolVLM-256M: 2.8 s per image on Jetson Orin; continuous VLM inference is impractical	One-pass decision model: 74 ms per image for four questions on an RTX 2080 Ti; about 0.67 s on a Jetson Orin NX (est.) (Section 4.3)	Partly: no Jetson measurement
Hierarchical processing	Proposes detectors that trigger the VLM; not implemented	Calibrated defect probability as the trigger: 15.3% of images passed on, 88.5% defect recall (Section 4.4)	Partly: computed offline, not deployed
Multimodal training data	GPT-4o captions for COCO-Bridge	Typed questions with labels from the dataset folders; no generated captions (Section 3.2)	Addressed for closed-set questions
Measured semantic quality	Three qualitative examples, pre-trained against fine-tuned	Macro F1, AUROC and calibration error with bootstrap intervals on 6,092 held-out images, zero-shot to fine-tuned (Sections 4.1–4.2)	Addressed
Trustworthy autonomy	Calls for confidence estimation and verification	Temperature-calibrated probabilities (ECE 0.058), a risk-based threshold, outputs restricted to known options; the model's own "judgeable" answer fails (Section 4.5)	Partly
Collaborative edge intelligence	Future direction	FedAvg over 3–18 tower-wise drones; a server-fine-tuned start for the fleet (Sections 4.6–4.7)	Partly: simulated drones
Communication of updates	Lists PEFT and LoRA for latency	Update size of each trainable part, 16.9–229.5 MB; full-model uplink 6.9–41.3 GB per run (Section 4.8)	Partly: small updates not yet trained
Cross-structure generalization	Defects differ across materials and structures	Test on 14 unseen towers; one asset type (power lines)	Partly
Energy, mission-aware scheduling, multimodal sensing, digital twins	Requirements and directions	Not studied	Open

is the image only. One predict call per image asks four questions:

- component (choice, 11 options): which power-line component is in the image.
- defect (yes/no): whether the component has a defect.
- condition (choice, 6 options): good, rust, corrosion, missing cap, bird nest, or other defect (torn part or peeling paint).
- judgeable (yes/no): whether the image is clear enough to judge. This question has no ground truth.

The image is encoded once and reused for the four questions. The model can only answer with one of the given options, so it cannot produce a description that contradicts the option set. It also cannot describe a defect outside that set.

3.2. Data and ground truth

ElecVQA-Bench [16] builds visual questions on the InsPLAD component crops [17]. Power-line inspection is one of the four domains of [1], and its anomaly types (degradation, missing component, corrosion) are the classes of this data. We use the tower14 split: 18 training, 10 validation and 14 test towers, with 39,228 items on 35,415 unique images. The towers in different splits share no images. Each image carries its component folder and its class folder. We take the component (17 folders, with the 7 shackle types merged into one, giving 11 classes) and the condition (6 classes, with torn-up and peeling-paint merged) from these folders. A defect is any condition other than good. The folder labels agree with every ElecVQA-Bench answer. The test towers hold 6,092 images, of which 601 (9.9%) show a defect. No caption is generated at any step, so the training labels need no teacher model and no manual text annotation.

3.3. Fine-tuning

We follow the fine-tuning guide of laya-vision [6]. Each image gives three labelled questions (component, defect, condition). Training uses the RLCD objective of Laya with soft cross-entropy, AdamW, batch 8, learning rates 10^{-4} for the head and 2×10^{-5} for the backbone, 25 warm-up steps and 2.5 passes over the training questions. Temperatures are refit on held-out training images after training [8]. The RTX 2080 Ti has no native bfloat16, so all training runs in fp32. We train three models:

- Round 1: up to 400 images per condition (1,893 images, 79% defects; 1,793 for training), training the head and the last 4 language layers (22.6M parameters).
- Round 2, last 4 layers: all 2,249 defect images of the training towers plus up to 300 good images per component (5,107 images, 44% defects; 4,807 for training, 300 for temperatures).

- Round 2, full: the same data, training all parameters except the vision encoder (114.7M parameters).

3.4. Federated fine-tuning

The simulated drones use the same 5,107 images as round 2, divided by tower. For K drones, towers are assigned to drones so that the image counts are balanced. With $K = 18$ each drone holds one tower, from 1 to 1,103 images, and four drones hold no defect image. In round r, the server sends the global weights θ^r to every drone. Drone k trains on its n_k questions for 0.25 passes with a new AdamW optimizer and returns θ_k^r. The server averages

$$\theta^{r+1} = \sum_{k=1}^{K} \frac{n_k}{n} \theta_k^r, \qquad n = \sum_k n_k. \tag{1}$$

Ten rounds give 2.5 passes in total, so every run uses the same number of training questions as central training. The learning rate follows a cosine decrease over the rounds. The trained part is the full model of round 2. Each drone keeps 6% of its images out of training. After the last round the drones send the logits and labels of these images, and the server fits the temperatures. No image leaves a drone. The simulation runs the drones one after another on one GPU, and the averaging in Eq. (1) is identical to the FedAvg strategy of Flower [12].

We run four kinds of experiment:

- Control: the same loop with one drone holding all images. It separates the effect of the loop from the effect of federation.
- FedAvg with $K = 3$, 6 and 18 drones.
- Local only: the six drones of $K = 6$ train alone for 2.5 passes each, with no averaging.
- Two starts: the 18 training towers are divided into a server group (13 towers, 2,554 images, 37% defects) and a fleet group (5 towers, 2,553 images, 51% defects), each with all components and conditions. A model is fine-tuned centrally on the server group. Then FedAvg with 5 fleet drones, one per tower, starts either from the published checkpoint or from the server model. This is the server–fleet split of the collaborative direction in [1]: the server trains with the data it may hold, and the fleet improves the model with data it may not share.

3.5. Evaluation

All numbers are on the test towers. For component and condition we report accuracy, balanced accuracy and macro F1, because the majority class covers 90% of the test images. For defect we report AUROC, average precision and expected calibration error (ECE). We tune a threshold on the validation towers that minimizes a risk of 10 per missed defect and 1 per false alarm, and report this risk per 1,000 images. Intervals are 95% bootstrap intervals over 1,000 resamples of the test images. Each configuration has one training seed.

Training sets with a different class mix than the test data shift the probabilities. As a correction that needs no retraining [18], we also report

$$\tilde{p}(c \mid x) \propto p(c \mid x) \, \frac{\pi_{\text{val}}(c)}{\pi_{\text{train}}(c)}, \tag{2}$$

where π are the label shares of the validation towers and of the training images. For comparison with earlier tools on the same benchmark, we also score the 7,057 test items of ElecVQA-Bench. A yes/no item takes $P(\text{defect}) \geq 0.5$, and a multiple-choice item takes the best condition among its options. The reference tools were evaluated on the same items. They are a ResNet-50 [19] classifier fine-tuned on the training towers, SmolVLM-256M tuned with LoRA [20] as in EVLM, and the same SmolVLM without tuning.

4. Results

4.1. Zero-shot

Without fine-tuning, laya-vision does not classify this data. Its condition accuracy is 0.834, below the 0.901 of always answering good. Its macro F1 is 0.180 (95% interval 0.172–0.190), against 0.158 for the majority answer. It calls 480 of the 601 defect images good, and the recall of corrosion, missing cap and bird nest is at most 0.02. The component accuracy is 0.141, below the majority baseline of 0.314. The defect question carries a weak signal (AUROC 0.714), with probabilities that are not calibrated (ECE 0.178). This puts a number on the qualitative finding of [1] that a pre-trained small VLM gives generic outputs on inspection images: here a general-purpose decision model is no better than a constant answer.

4.2. Fine-tuning

Figure 1 and Table 2 show the progression. Round 1 lifts the defect AUROC to 0.898. Its plain accuracy falls because the model learned a training mix with 79% defects and labels 1,971 of 5,491 good test images as defective. The correction in Eq. (2) raises its condition accuracy from 0.642 to 0.856 and its macro F1 from 0.303 to 0.451. Round 2 adds data, covers all components and has 44% defects. It improves every metric, and the full model is the best: condition accuracy 0.952 (above the majority answer), macro F1 0.703, component accuracy 0.924, defect AUROC 0.965 and ECE 0.058. For this model the correction changes little (condition accuracy 0.957).

Table 3 gives the per-condition result of the full model. Bird nests are always found. Rust has the most false alarms: 152 good images are called rust, so rust precision is 0.62. The 5 test images of other defects are always missed. Among the components, the spacer is the only one that the model does not recognize (recall 0.06, with

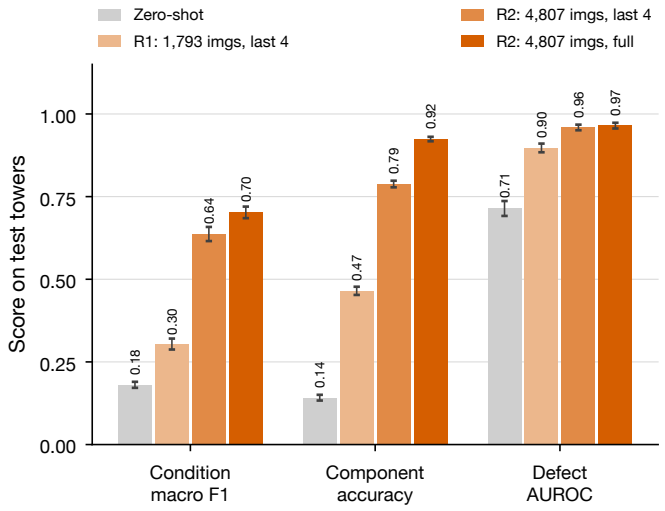


Figure 1: More data and a fully trained backbone lift every metric. Test-tower scores of laya-vision from zero-shot to the round-2 full fine-tune (6,092 images). Whiskers are 95% bootstrap intervals over images; one training seed per model; no class-share correction.

Table 2: Test-tower results. Images: 6,092 test images. Items: 7,057 ElecVQA-Bench test items. The intervals for laya-vision are in the text and in Fig. 1. Reference tools are scored on the same items.

Model	Cond. macro F1	Comp. acc.	Defect AUROC	Item acc.
Zero-shot	0.180	0.141	0.714	0.830
Round 1, last 4	0.303	0.465	0.898	0.702
Round 2, last 4	0.637	0.789	0.959	0.923
Round 2, full	0.703	0.924	0.965	0.948
ResNet-50, tuned	–	–	–	0.961
SmolVLM + LoRA	–	–	–	0.956
SmolVLM, no tuning	–	–	–	0.830

18 training images); all other components have a recall of at least 0.87.

On the ElecVQA-Bench items the full model reaches 0.948, below the fine-tuned ResNet-50 (0.961) and SmolVLM with LoRA (0.956). The decision model therefore trades about one point of item accuracy against the generative SmolVLM of EVLM for a single forward pass. The full model trains for 24 minutes on an RTX 2080 Ti with a peak memory of 3.6 GB. In training, most batches after step 1,600 reach the maximum reward, and the refit temperatures are large (4.1 for choice questions and 6.2 for yes/no questions). The test results remain close to the validation results (condition accuracy 0.952 against 0.935, defect AUROC 0.965 against 0.968), so the model does not overfit the training towers.

4.3. Latency

On the RTX 2080 Ti in fp32, one predict call with all four questions takes a median of 72.0 ms per image (p95 98.5 ms), including reading and preprocessing the image. The fine-tuned full model takes 74.0 ms. The condition question alone takes 71.4 ms, so almost all of the time is spent encoding the image, and further questions are nearly free. Peak GPU memory for inference is 0.9 GB. With 3.45 key frames per 4 s segment, as in EVLM [2], one segment needs 0.25 s. On one thread of the server CPU, one image takes 2.5 s.

Table 3: Per-condition test results of the round-2 full model (6,092 test images, threshold-free argmax over the six conditions).

Condition	Images	Precision	Recall	F1
Good	5,491	0.986	0.965	0.975
Rust	308	0.618	0.834	0.710
Corrosion	110	0.833	0.818	0.826
Missing cap	129	0.743	0.783	0.762
Bird nest	49	0.891	1.000	0.942
Other defect	5	0.000	0.000	0.000

We have no Jetson measurement. We estimated a factor of 9 between a Quadro GV100 and a Jetson Orin NX from their fp32 throughput and memory bandwidth. The RTX 2080 Ti has a similar fp32 throughput to the GV100, so the same factor gives about 0.67 s per image and 2.2 s per 4 s segment; these are estimates. On a Jetson Orin with the same number of CUDA cores as the Orin NX, [1] measured 2.8 s per image for SmolVLM-256M, and EVLM reports 1.4–6.4 s per segment [2]. If our estimate holds, the decision model answers four questions about 4 times faster than the describing model takes for one caption, and a segment fits inside its own 4 s duration. The comparison is approximate: the hardware is not identical, the other systems generate text while laya-vision returns labels, and EVLM processes video segments while we process single crops.

4.4. The decision model as a semantic trigger

The agenda of [1] calls for hierarchical pipelines in which an inexpensive stage decides which frames reach the VLM, and proposes a detector for this stage. A detector localizes components but does not judge their condition. The decision model does judge the condition, so it can serve as the trigger itself. We use the defect threshold tuned on the validation towers (0.057). On the test towers it passes 931 of 6,092 images (15.3%) to a following stage and finds 532 of the 601 defects (recall 0.885), with a false-alarm rate of 0.073 among good images and a risk of 179 per 1,000 images, against 987 for always answering good. These rates are computed from the per-image predictions; the cascade itself is not deployed.

A rough time budget follows if this trigger runs on every frame and a describing model runs only on the passed frames. For the 300-frame clip of [1], 300 decision-model calls at an estimated 0.67 s and 46 descriptions at the measured 2.8 s take about 5.5 min, against 14 min for describing every frame. Both stages are slower than the 30 fps camera, so the key-frame filter of EVLM is still needed in front. This budget mixes our estimate with their measurement and is meant only to show the order of the saving.

4.5. Calibration and confidence

A mission planner can act on a probability only if the probability is reliable. After fine-tuning, the defect probability has an ECE of 0.058 (0.178 zero-shot) and an average precision of 0.862. The defect answer and the condition answer agree on 97.5% of the test images, which gives a second, independent check. The judgeable question, which asks the model whether it can judge the image, does not work as a confidence signal: its AUROC for a correct condition answer is 0.53 zero-shot and 0.31 after fine-tuning. Confidence for a decision model should therefore come from its calibrated answer probabilities, not from a self-assessment question.

4.6. Federated fine-tuning across drones

The control with one drone matches central training: macro F1 0.697 against 0.703 (Table 4). The intervals overlap for all three metrics, so the training loop does not cause the differences below. With the same data spread over more drones, FedAvg loses accuracy (Fig. 2b, c). From 1 to 18 drones, the condition macro F1 falls from 0.697 to 0.474 and the component accuracy from 0.933 to 0.700. The defect AUROC degrades less, from 0.959 to 0.933, so the trigger of Section 4.4 is the most robust output under federation. The validation curves (Fig. 2a) show that the 6- and 18-drone runs are still improving after ten rounds, at 0.839 and 0.826, while the control has levelled off at 0.938. Every run receives the same total training, and the server averages updates that each cover only a part of the data. We therefore expect, but have not tested, that more rounds would close part of the gap.

Federation still helps the individual drone. With six drones, FedAvg is better than the average drone that trains alone on every metric: macro F1 0.541 against 0.371, component accuracy 0.761 against 0.621, defect AUROC 0.939 against 0.881. It is also better than the best single drone in defect AUROC (0.939 against 0.916). The best single drone has a macro F1 of 0.525, inside the FedAvg interval, and a slightly higher component accuracy (0.782 against 0.761). Drones with few defects do worst alone: a drone with 29% defects reaches a macro F1 of 0.278.

4.7. A fine-tuned start for the fleet

Figure 3 compares the two starts on the same fleet. From the published checkpoint, fleet FedAvg reaches a macro F1 of 0.513, a component accuracy of 0.593 and a defect AUROC of 0.932. From the server model it reaches 0.636, 0.912 and 0.953, and the intervals of the two starts do not overlap. The fleet also improves the server model, from 0.558 to 0.636 macro F1 and from 0.903 to 0.953 AUROC, while its images stay on the drones. The fine-tuned start is already at 0.904 validation accuracy before the first round and ends at 0.939. Its item accuracy (0.937) is close to that of central training on all 18 towers (0.948). For the collaborative direction of [1], this gives a concrete recipe: fine-tune on whatever data the operator may centralize, then let the fleet continue with FedAvg.

4.8. Communication

The full-model update is 229.5 MB per drone and round in fp16, so a run needs 6.9 GB (3 drones) to 41.3 GB (18 drones) of uplink in total. Uploading all 5,107 training images once at about 35 kB each would take about 0.18 GB. Federation therefore does not save bandwidth in this setting. Its benefit is that the images stay on the drones. The updates are too large for a 4G uplink during a mission, so they would be sent from the ground station after a mission.

The agenda of [1] lists parameter-efficient fine-tuning as a means for edge deployment. In federated training it is also the means to reduce the uplink, because the trained part is the part each drone sends. Table 5 gives the size of each choice, counted from the checkpoint. The vision encoder (86.4M parameters) is frozen in every choice. Token embeddings are a quarter of the full update, although the questions and options never change. The decision head alone is 16.9 MB, and with LoRA of rank 8 on all language layers the update is 20.2 MB, about 11 times smaller than the full update. For scale, EVLM sends 3.75 MB of results for a 10-minute mission [2]. We have not yet trained these smaller updates, so their accuracy is open.

5. Discussion

What the decision model adds to semantic edge inspection. The agenda of [1] frames semantic inspection as description. Our results show that a large part of the inspection decision does not need description. With the same SmolVLM backbone family, a decision model answers component, defect and condition questions with measured accuracy close to a fine-tuned describing model, from one image encoding. It removes the need for synthetic captions, because the labels already exist in the dataset structure, and it gives probabilities that can drive a trigger. The describing VLM remains useful for the report on the frames the trigger passes, and for defects outside the option set.

What federation adds. The collaborative direction of [1] becomes testable once the onboard model is a classifier with a fixed output set. FedAvg across tower-wise drones beats training alone, and a server-fine-tuned start brings the fleet close to central training. The cost is accuracy as the data is split further under a fixed budget, and a full-model uplink of several gigabytes per run.

Limitations. Each configuration has one seed. The drones are simulated on one GPU, so update sizes and uplink are calculated, not measured. The Jetson latency is an estimate from a hardware factor, and the comparison with [1] and EVLM compares different outputs on hardware that is not identical. We do not measure energy; [1] found energy modest relative to the battery, so latency is the more important constraint, but this should be checked for the decision model. The fixed compute budget is fair for compute but gives FedAvg with many drones fewer effective

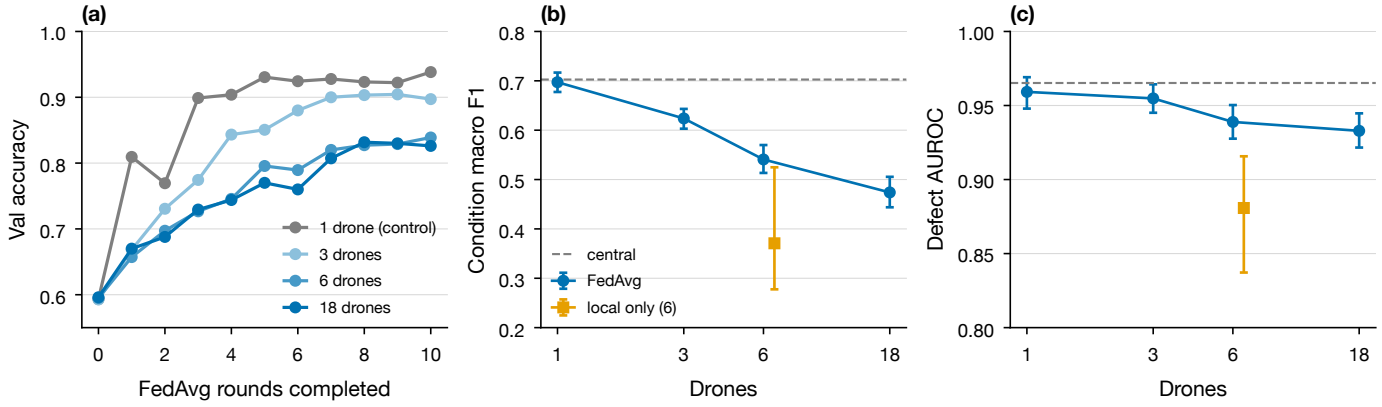


Figure 2: Splitting the same data over more drones slows FedAvg. (a) Validation accuracy on 600 images after each round. (b, c) Test-tower condition macro F1 and defect AUROC against the number of drones. FedAvg whiskers are 95% bootstrap intervals. The local-only marker is the mean of six drone models, and its whisker spans their minimum to maximum. The dashed line is central training. Every run uses the same total training (2.5 passes), with one seed per run.

Table 4: Federated runs on the test towers, with 95% bootstrap intervals. Local only: mean (minimum–maximum) of the six drone models. Uplink: full-model update in fp16 (229.5 MB) × drones × 10 rounds, calculated, not measured.

Run	Cond. macro F1 (95% interval)	Comp. acc.	Defect AUROC	Uplink
Central (round 2, full)	0.703 (0.685–0.720)	0.924	0.965	–
Control, 1 drone	0.697 (0.677–0.717)	0.933	0.959	–
FedAvg, 3 drones	0.624 (0.603–0.643)	0.874	0.955	6.9 GB
FedAvg, 6 drones	0.541 (0.513–0.570)	0.761	0.939	13.8 GB
FedAvg, 18 drones	0.474 (0.444–0.505)	0.700	0.933	41.3 GB
Local only, 6 drones	0.371 (0.278–0.525)	0.621	0.881	–
Server towers, central	0.558 (0.532–0.584)	0.894	0.903	–
Fleet FedAvg, base start	0.513 (0.484–0.540)	0.593	0.932	11.5 GB
Fleet FedAvg, fine-tuned start	0.636 (0.612–0.657)	0.912	0.953	11.5 GB

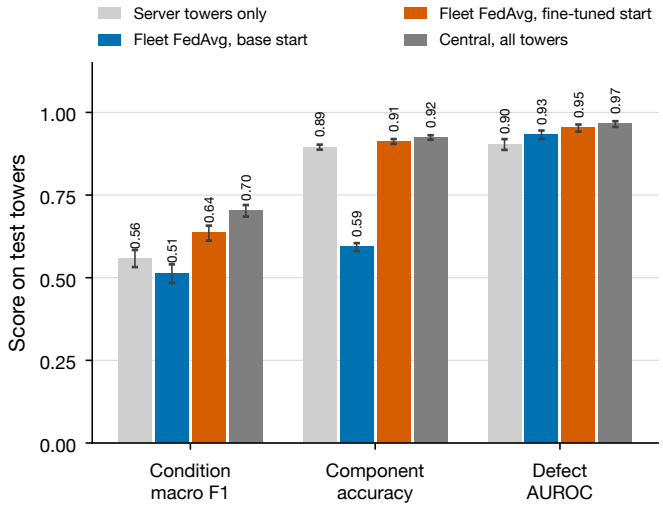


Figure 3: A fine-tuned start makes fleet FedAvg close to central training. The server model is trained on 13 towers. The fleet has 5 towers, one drone each, and starts either from the published checkpoint (base) or from the server model. Central training uses all 18 towers. Whiskers are 95% bootstrap intervals; one seed per run.

steps. The data comes from one benchmark and one asset type, with few images in some classes (5 test images of other defects, 17 spacers). The tower split groups images by location, but the drones are not real flights. The trigger is evaluated offline on crops, not in a deployed cascade.

Table 5: Trainable part of laya-vision and the size of one drone update (parameters counted from the checkpoint; 2 bytes per parameter in fp16, 1 in int8). Only the full update is evaluated in this paper.

Trained and sent	Parameters	fp16	int8
Decision head	8.46M	16.9 MB	8.5 MB
Head + LoRA rank 8	10.09M	20.2 MB	10.1 MB
Head + last 4 layers	22.62M	45.2 MB	22.6 MB
Head + 20 language layers	79.27M	158.5 MB	79.3 MB
All but vision encoder	114.73M	229.5 MB	114.7 MB

Future work. We plan to complete the open rows of Table 1: a Jetson measurement of latency and energy; federated rounds with the smaller updates of Table 5, int8 updates and FedProx [11] for drones without defect images; longer FedAvg runs with fixed local steps; few labels per drone, drawn with the natural defect share; drones that see different components; more seeds; a deployed cascade with a describing VLM behind the trigger; and a second asset type, such as bridges, for cross-structure generalization.

6. Conclusion

The edge-native semantic inspection agenda [1] leaves latency, trigger design, training data, confidence and fleet collaboration open. A 201M-parameter vision-language decision model addresses part of each. It fails zero-shot on power-line component images, but after fine-tuning on folder labels it reaches 0.948 item accuracy, close to fine-tuned ResNet-50 and SmolVLM tools, answers four questions in 74 ms per image on a desktop GPU, and gives calibrated defect probabilities that pass 15.3% of images on while finding 88.5% of the defects. FedAvg across tower-wise drones beats training alone, and a start fine-tuned on separate server towers brings the fleet close to central training while the images stay on the drones.

Data and code availability

The code, the per-image predictions and the evaluation files are in the project repository, branch vision-fl (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [16, 17]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [21].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL, arXiv:2605.18853 (2026).

[8] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[10] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[12] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[13] Askin, et al., Federate the router, arXiv:2601.22318 (2026).

[14] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[15] From open-vocabulary detection to structural assessment: An agentic federated approach for guardrail monitoring (2026). https://www.sciencedirect.com/science/article/pii/S2772503026000459.

[16] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[17] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[18] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[21] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.